EGC Covoiturage

Document de conception — Architecture & Produit

Version 2.0 · **Date** Juin 2026 **Client** École de Gymnastique — EGC **Statut** MVP fonctionnel — en cours de validation

---

Table des matières

1. Contexte et objectifs
2. Vision produit
3. Utilisateurs et rôles
4. Cas d'usage
5. Architecture technique
6. Modèle de données
7. Sécurité
8. UX / Parcours utilisateurs
9. Fonctionnalités avancées
10. Roadmap produit
11. Infrastructure et coûts
12. Recommandations

Changelog

Version	Date	Modifications principales
1.0	Mai 2026	Document initial
2.0	Juin 2026	Inscription parents, onglet notifications, formatage téléphone/date/heure, ciblage groupes, feature flags, corrections RLS, dépôt Git

---

1. Contexte et objectifs

Contexte

L'EGC est une association de gymnastique comptant environ **1 600 licenciés**, principalement des enfants et leurs parents. L'organisation des trajets vers les entraînements et compétitions repose aujourd'hui sur des échanges informels (téléphone, WhatsApp), ce qui génère une charge logistique importante pour les familles et le club.

Problème à résoudre

- Pas d'outil centralisé pour coordonner les covoiturages
- Risque de double sollicitation ou d'oubli

- Aucune traçabilité des trajets impliquant des enfants mineurs
- Charge administrative pour les coordinateurs du club

Objectifs produit

Objectif	Indicateur de succès
Simplifier l'organisation des trajets	Temps moyen pour trouver un trajet < 3 minutes
Mettre en relation rapidement les membres	Taux de réservation acceptée > 70 %
Réduire la charge logistique	Réduction des appels/messages organisationnels
Créer une solution sécurisée	0 incident lié à un accès non autorisé

---

2. Vision produit

Positionnement

EGC Covoiturage est une application mobile fermée de mise en relation pour le transport entre membres d'un club sportif. Ce n'est pas un service de transport public ouvert : c'est un **outil de coordination interne**, extension numérique du lien de confiance existant au sein du club.

> « Trouver un covoiturage vers le gymnase aussi simplement qu'envoyer un message. »

Proposition de valeur

Pour	EGC Covoiturage permet de
Les parents conducteurs	Organiser leurs trajets et remplir leur voiture avec des enfants connus du club
Les parents passagers	Trouver un trajet fiable en moins de 3 taps, sans appels ni messages WhatsApp
Le club	Réduire la charge logistique et garder le contrôle sur les déplacements
Les administrateurs	Gérer les comptes de façon centralisée et sécurisée

Différenciateurs clés

- **Accès fermé** : aucun inconnu, tous les membres sont vérifiés par le club
- **Confiance par défaut** : pas d'évaluation par étoiles nécessaire, la communauté est déjà établie
- **Adapté aux enfants** : logique de sécurité spécifique aux mineurs (confirmation renforcée, traçabilité)
- **Simple** : pensé pour des parents non techniques, max 3 taps pour une réservation

Principes de design

1. **Zéro friction** — un parent doit pouvoir trouver et réserver un trajet en moins de 3 interactions
2. **Confiance visible** — chaque profil est nominatif, pas d'anonymat
3. **Mobile-first** — optimisé pour une utilisation rapide depuis un téléphone
4. **Progressive disclosure** — les fonctions avancées ne s'affichent que si pertinentes

---

3. Utilisateurs et rôles

Types de comptes

Rôle	Création du compte	Identifiants	Droits
Licencié	Par l'administrateur uniquement	N° de licence + date de naissance	Consulter et réserver des trajets
Parent	Inscription via formulaire (validée par admin)	Email + mot de passe	Proposer et réserver des trajets, associer des enfants
Encadrant	Inscription libre	Email + mot de passe	Mêmes droits que Parent
Administrateur	Créé manuellement en base	Email + mot de passe fort	Accès complet, gestion des comptes et événements

Règles spécifiques

- Les comptes **licenciés sont créés uniquement par l'admin** (via formulaire ou import CSV)
- Le mot de passe initial d'un licencié est sa **date de naissance** (à modifier lors de la 1re connexion)
- Les comptes **parents sont créés via un formulaire d'inscription** et restent inactifs (`is_active = false`) jusqu'à validation par un admin
- L'inscription parent nécessite de **fournir le numéro de licence d'un enfant** du club pour vérifier l'appartenance
- La relation parent-enfant doit être **validée par un admin** avant activation
- Les admins peuvent **suspendre et supprimer** tout compte
- Toute action admin est **tracée dans un journal d'audit**

Profil utilisateur

Champs obligatoires

- Prénom, Nom
- Téléphone (format `XX XX XX XX XX`, formaté automatiquement)
- Adresse (géocodée pour le calcul de distance, sélection obligatoire depuis la liste d'autocomplétion)
- Rôle

Champs conditionnels

- Email : obligatoire pour parents/encadrants, renseigné lors de la 1ère connexion pour les licenciés
- Nombre de places dans la voiture (liste déroulante 0–5, 0 = pas de véhicule)

Champs optionnels

- Groupe d'entraînement (pour les licenciés)

---

4. Cas d'usage

4.1 Covoiturage régulier

Contexte : trajets récurrents vers les entraînements hebdomadaires.

- Le conducteur crée un trajet de type **Régulier** avec une règle de récurrence (ex : tous les lundis à 18h)
- Les passagers réservent une fois, la réservation s'applique automatiquement chaque semaine
- Le conducteur reçoit une notification de confirmation chaque semaine

4.2 Covoiturage ponctuel

Contexte : besoin exceptionnel, remplacement, imprévu.

- Publication rapide en moins de 2 minutes
- Notification automatique aux membres du même groupe
- Réservation en 1 tap

4.3 Covoiturage événement / compétition

Contexte : déplacement vers une compétition, parfois avec nuitée.

- L'admin crée l'événement (nom, lieu, date, infos hébergement si multi-jours)
- L'admin **cible les groupes concernés** : seuls les membres de ces groupes sont notifiés
- Les conducteurs proposent des trajets **Aller** et/ou **Retour** séparément
- Pour les événements multi-jours : confirmation renforcée avec affichage du conducteur et son téléphone
- L'admin dispose d'un tableau de synthèse enfant/conducteur pour chaque événement

4.4 Inscription parent (nouveau)

Contexte : un parent souhaite rejoindre l'application.

- Le parent remplit un formulaire complet (nom, prénom, email, mot de passe, téléphone, adresse)
- Il saisit le **numéro de licence d'un de ses enfants** pour prouver son appartenance au club
- Le système vérifie en temps réel que ce numéro existe (via fonction SQL sécurisée, sans authentification requise)
- La demande est envoyée à l'admin ; le compte reste **inactif jusqu'à validation manuelle**
- L'admin reçoit une notification in-app et valide/refuse depuis la page de gestion des membres

4.5 Re-réservation après annulation

Contexte : un passager annule une réservation puis souhaite se réinscrire.

- L'application gère ce cas via un **UPSERT** (remplacement de la réservation annulée)
- Aucun message d'erreur de contrainte unique ne remonte à l'utilisateur

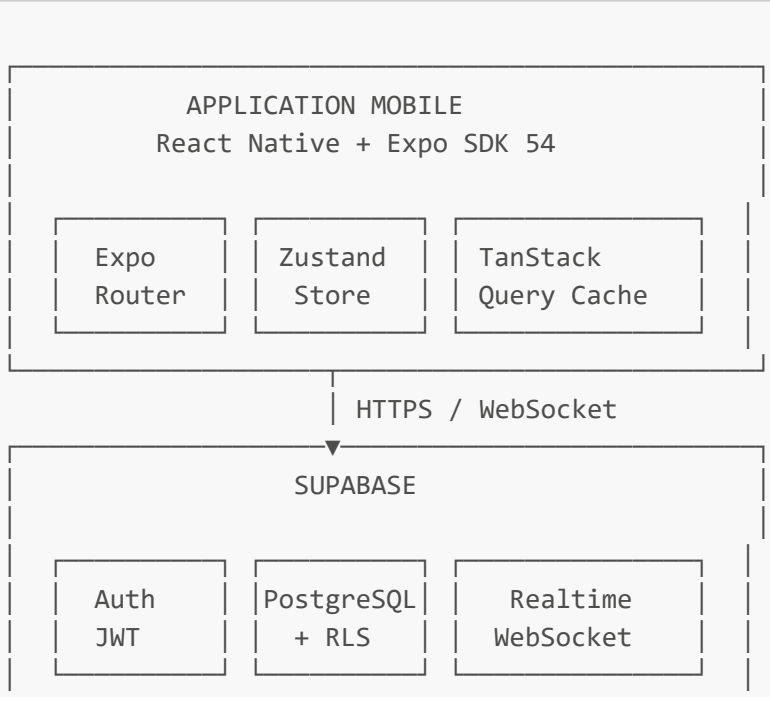
---

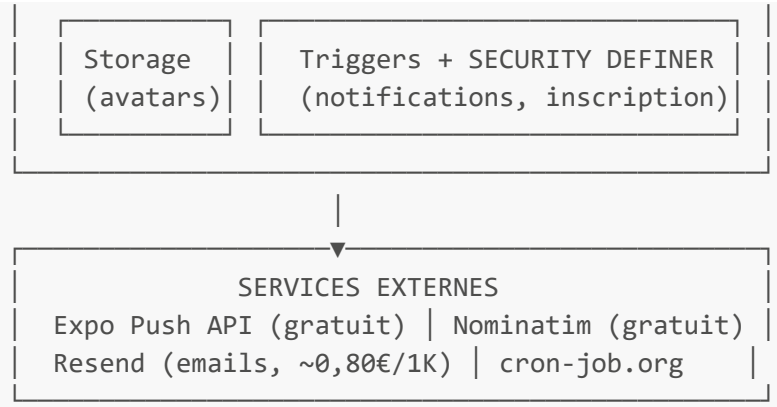
5. Architecture technique

Stack technologique

Composant	Technologie	Justification
Mobile	React Native + Expo SDK 54	Cross-platform iOS/Android, une seule base de code
Navigation	Expo Router 6.x	File-based routing, deep links natifs, TypeScript
État global	Zustand	Léger, simple, sans boilerplate
Cache données	TanStack Query	Gestion cache, requêtes, invalidation
Backend	Supabase (Free tier)	BaaS managé : PostgreSQL, Auth, Realtime, Storage
Base de données	PostgreSQL (via Supabase)	Relationnel, RLS natif, robuste
Authentification	Supabase Auth (JWT)	Flows custom supportés, tokens rotatifs
Messagerie temps réel	Supabase Realtime (WebSocket)	Inclus dans Supabase, pas de serveur custom
Notifications push	Expo Push Notifications	Gratuit, intégration native Expo
Cartes	react-native-maps + OpenStreetMap	Gratuit, sans API key
Géocodage	Nominatim (OpenStreetMap)	Gratuit, open source, sans limite critique
Emails transactionnels	Resend + pg_net (PostgreSQL)	~0,80€/1 000 emails, désactivable via feature flag
Monitoring	Sentry (Free tier)	Remontée automatique des erreurs
Builds	Expo EAS (Free tier)	30 builds/mois, suffisant pour un club

Vue d'ensemble architecture





Structure des fichiers

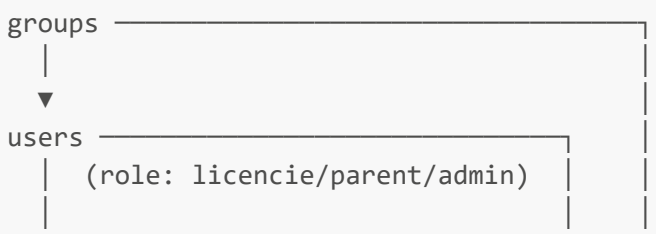
```
EGC/
├── app/                              # Écrans (Expo Router)
│   ├── _layout.tsx                   # Root layout + Auth Gate
│   ├── (auth)/                       # Écrans non authentifiés
│   │   ├── login.tsx                 # Connexion (licencié / parent)
│   │   ├── register.tsx              # ★ Inscription parent (nouveau)
│   │   └── complete-profile.tsx      # Complétion profil 1ère connexion
│   ├── (app)/                        # Écrans membres
│   │   ├── index.tsx                 # Accueil
│   │   ├── trips/
│   │   │   ├── index.tsx             # Liste des trajets (filtres: Ponctuel,
Régulier)
│   │   │   ├── [id].tsx              # Détail + réservation + messagerie
conducteur
│   │   │   └── create.tsx            # Création (3 étapes : Départ/Arrivée,
Détails, Récap)
│   │   ├── messages/
│   │   │   ├── index.tsx             # Liste des conversations
│   │   │   └── [id].tsx              # Chat temps réel
│   │   ├── notifications/
│   │   │   └── index.tsx             # ★ Onglet notifications (nouveau)
│   │   └── profile.tsx               # Profil + édition (nom, téléphone, adresse,
places)
│   └── (admin)/                      # Interface administrateur
│       ├── index.tsx                 # Tableau de bord + déconnexion
│       ├── users.tsx                 # ★ Demandes en attente + gestion membres
│       └── events.tsx                # ★ Événements avec ciblage par groupes
│
├── src/
│   ├── components/
│   │   ├── ui/
│   │   │   ├── Button.tsx
│   │   │   ├── Input.tsx
│   │   │   ├── Card.tsx
│   │   │   ├── Avatar.tsx
│   │   │   ├── Badge.tsx
│   │   │   ├── AddressInput.tsx      # Autocomplétion Nominatim (sélection
```

```
obligatoire)
│   │   │   └── DatePickerInput.tsx   # ★ Calendrier custom (grille
jours/mois/année)
│   │   ├── TripCard.tsx
│   │   └── MessageBubble.tsx
│   ├── constants/
│   │   ├── theme.ts                  # Couleurs, spacing, typographie
│   │   └── features.ts               # ★ Feature flags (fonctionnalités
payantes)
│   ├── hooks/
│   │   ├── useTrips.ts
│   │   ├── useMessages.ts
│   │   └── useBookings.ts            # UPSERT pour re-réservation après
annulation
│   ├── lib/
│   │   ├── supabase.ts               # Client Supabase + helpers auth
│   │   ├── notifications.ts          # Expo Push
│   │   ├── geocoding.ts              # Nominatim
│   │   └── formatters.ts             # ★ Formatage auto date, heure, téléphone
│   ├── stores/
│   │   └── auth.store.ts             # État d'authentification global
│   └── types/
│       └── index.ts                  # Interfaces TypeScript
│
├── supabase/
│   └── migrations/                   # Scripts SQL à exécuter dans l'ordre
│       ├── 001_initial_schema.sql    # Schéma complet
│       ├── 002_rls_policies.sql      # Row Level Security
│       ├── 003_functions_triggers.sql # Triggers notifications push
│       ├── 004_email_notifications.sql # Emails via Resend + pg_net
│       ├── 005_event_groups.sql      # ★ Ciblage groupes sur événements
│       ├── 006_verify_license.sql    # ★ Vérification licence (SECURITY DEFINER)
│       ├── 007_register_parent.sql   # ★ Inscription parent (SECURITY DEFINER)
│       └── 008_feature_flags.sql     # ★ Flags fonctionnalités payantes
│
├── .env                              # Clés Supabase (non commité)
├── .env.example                      # Modèle de configuration
├── .gitignore                        # node_modules, .env, .expo, dist
└── .gitattributes                    # Gestion fins de ligne LF/CRLF
```

---

6. Modèle de données

Schéma relationnel simplifié



```
    ├── parent_child (parent→enfant)  │     │
    │                                 │     │
    ├── trips (as driver) ────────────┼─────┘
    │     │                           │
    │     ├── bookings ←──────────────┘ (as passenger)
    │     │
    │     └── conversations
    │               │
    │               └── messages
    │
    └── notifications

  events (group_ids[]) ──→ trips
  app_settings (feature flags)
```

Tables principales

`users` — Membres du club

Colonne	Type	Description
`id`	UUID	Identifiant interne
`auth_id`	UUID	Lié à Supabase Auth
`role`	ENUM	licencie / parent / encadrant / admin
`license_number`	TEXT	Numéro de licence (licenciés uniquement)
`email`	TEXT	Email (parents/encadrants + licenciés après 1ère connexion)
`first_name`	TEXT	Prénom
`last_name`	TEXT	Nom
`phone`	TEXT	Téléphone — **obligatoire** (format `XX XX XX XX XX`)
`address`	TEXT	Adresse domicile
`lat` / `lng`	DECIMAL	Coordonnées géocodées
`group_id`	UUID	Groupe d'entraînement
`car_capacity`	SMALLINT	Nombre de places (0 = pas de voiture)
`is_active`	BOOLEAN	Compte actif / suspendu / en attente de validation
`profile_complete`	BOOLEAN	Profil complété lors de la 1ère connexion
`push_token`	TEXT	Token Expo Push Notifications

`trips` — Trajets

Colonne	Type	Description
`id`	UUID	Identifiant
`driver_id`	UUID	Conducteur
`event_id`	UUID	Événement associé (optionnel)
`type`	ENUM	**ponctuel / regulier** (Compétition supprimé de l'interface)
`direction`	ENUM	aller / retour (pour les événements)
`status`	ENUM	actif / complet / annule / termine
`departure_address`	TEXT	Adresse de départ (Mon adresse ou saisie libre)
`arrival_address`	TEXT	Adresse d'arrivée
`departure_time`	TIMESTAMPTZ	Date et heure de départ
`available_seats`	SMALLINT	Places proposées (1–9)
`booked_seats`	SMALLINT	Places réservées (calculé automatiquement)
`comment`	TEXT	Note du conducteur
`recurrence_rule`	TEXT	Règle iCal pour trajets réguliers

`bookings` — Réservations

Colonne	Type	Description
`id`	UUID	Identifiant
`trip_id`	UUID	Trajet concerné
`passenger_id`	UUID	Passager
`status`	ENUM	en_attente / accepte / refuse / annule
`seats_requested`	SMALLINT	Nombre de places demandées
`message`	TEXT	Message d'accompagnement optionnel

Contrainte unique `(trip_id, passenger_id)` — les re-réservations utilisent un **UPSERT** pour écraser le statut `annule` sans erreur.

`events` — Compétitions et événements

Colonne	Type	Description
`id`	UUID	Identifiant
`name`	TEXT	Nom de l'événement
`event_date`	DATE	Date de début

Colonne	Type	Description
`end_date`	DATE	Date de fin (si multi-jours)
`is_multi_day`	BOOLEAN	Événement sur plusieurs jours
`location`	TEXT	Lieu
`accommodation_note`	TEXT	Informations hébergement si nuitée
`group_ids`	UUID[]	★ Groupes ciblés (vide = tous les membres)

`app_settings` — Configuration et feature flags

Clé	Valeur par défaut	Description
`club_address`	—	Adresse du club EGC
`resend_api_key`	—	Clé API Resend (emails transactionnels)
`email_from`	—	Expéditeur des emails
`email_notifications_enabled`	`true`	★ Active/désactive les emails Resend
`sms_notifications_enabled`	`false`	★ Active/désactive les SMS (non implémenté)

`conversations` + `messages` — Messagerie

Une conversation est liée à un trajet et deux utilisateurs. Les messages sont chargés et livrés en temps réel via Supabase Realtime (WebSocket). L'écran de chat intègre un `KeyboardAvoidingView` pour éviter que le clavier masque la zone de saisie.

`parent_child` — Lien parent-enfant

Colonne	Type	Description
`parent_id`	UUID	ID du parent (auth_id Supabase)
`child_id`	UUID	ID du licencié (users.id)
`validated`	BOOLEAN	Validé par un admin

---

7. Sécurité

Authentification

Licencié

```
Login    : numéro de licence
Password : date de naissance (format YYYY-MM-DD)
Email interne généré : {licence}@egc.internal (jamais visible par l'utilisateur)
```

Parent / Encadrant

```
Login    : adresse email
Password : mot de passe choisi (min 8 caractères)
Compte créé via formulaire d'inscription, activé par l'admin uniquement
Confirmation email : désactivée (validation manuelle suffit, évite la limite de
3/h)
```

Administrateur

```
Compte créé manuellement en base
MFA recommandé
Tokens JWT avec expiration courte (1h) + refresh tokens rotatifs
```

Row Level Security (RLS)

Toutes les tables ont le RLS activé. Les règles garantissent :

- Un utilisateur **ne voit que les trajets actifs** (pas les annulés sauf les siens)
- Un utilisateur **ne voit que ses propres messages et conversations**
- Un passager **ne peut pas réserver son propre trajet**
- Seuls les **admins peuvent créer des comptes licenciés**
- Seuls les **admins peuvent suspendre/supprimer** des comptes
- Les **logs d'audit sont en lecture seule** pour les non-admins

Fonctions SECURITY DEFINER

Certaines opérations nécessitent de contourner le RLS de façon contrôlée. Ces fonctions s'exécutent avec les droits du propriétaire de la base et incluent leurs propres vérifications de sécurité :

Fonction	Usage	Contrôle de sécurité
`verify_license_exists(p_license)`	Vérifier qu'un numéro de licence existe avant inscription (utilisateur anonyme)	Aucune donnée sensible exposée (retourne uniquement exists + prénom/nom)
`register_parent_account(...)`	Créer le profil parent + lien enfant + notifier admins en une seule transaction atomique	Vérifie que `auth.uid() = p_auth_id` avant toute insertion

Fonction	Usage	Contrôle de sécurité
`notify_new_event_to_groups()`	Notifier les membres des groupes ciblés lors d'un nouvel événement	Trigger SQL, pas d'appel externe possible
`send_email(...)`	Envoyer un email via Resend + pg_net	Vérifie `email_notifications_enabled` avant l'envoi

Mesures de sécurité complémentaires

Mesure	Détail
HTTPS uniquement	Certificat TLS géré par Supabase
Rate limiting	Anti brute-force sur les endpoints de login
Données enfants non exposées	Les noms d'enfants ne sont jamais dans les listes publiques
Téléphone masqué	Visible uniquement après confirmation d'une réservation
Signalement	Bouton sur chaque trajet/conversation → notification admin immédiate
Conformité RGPD	Droit à l'effacement, pas de données sensibles stockées en clair
Adresses @egc.internal	Les licenciés n'ont jamais conscience de leur email interne

---

8. UX / Parcours utilisateurs

Navigation principale

```
┌──────────────────────────────────────────────────────┐
│                   BARRE DE TABS                      │
│  [Accueil]  [Trajets]  [Messages]  [Notifs]  [Profil]│
└──────────────────────────────────────────────────────┘
```

Cinq onglets visibles. Écrans cachés accessibles par navigation : `trips/[id]`, `trips/create`, `messages/[id]`.

Parcours 1 — Connexion

```
Écran de login
    │
    ├── [Licencié] ──→ N° de licence + Date de naissance (calendrier custom)
    │                          │ (succès)
    │                          ▼
    │                  Auth Gate vérifie profil
    │                          │
```

```
    │                          ├── profil incomplet ──→ complete-profile.tsx
    │                          └── profil complet   ──→ (app)
    │
    └── [Parent/Encadrant] ──→ Email + Mot de passe
                                    │ (pas de compte ?)
                                    ▼
                               "Faire une demande" ──→ register.tsx
```

Parcours 2 — Inscription parent (nouveau)

```
register.tsx
    │
    ├── Saisie : Prénom, Nom, Email, Mot de passe (8 car. min)
    ├── Téléphone (formaté automatiquement : XX XX XX XX XX)
    ├── Adresse (autocomplétion Nominatim, sélection obligatoire)
    │
    └── Vérification d'appartenance
            │ Numéro de licence d'un enfant
            │ → RPC verify_license_exists() [SECURITY DEFINER, anonyme]
            │
            ├── licence inconnue → erreur inline
            └── licence valide
                    │
                    ▼
            supabase.auth.signUp()
                    │
                    ▼
            RPC register_parent_account() [SECURITY DEFINER]
                ├── INSERT users (is_active = false)
                ├── INSERT parent_child (validated = false)
                └── INSERT notifications (admins)
                    │
                    ▼
            Écran de succès
            "Votre demande a été transmise à l'administrateur"
```

Parcours 3 — Première connexion (Licencié)

```
complete-profile.tsx — Étape 1 / 2
    ├── Email (renseigné ici, obligatoire pour les licenciés)
    ├── Adresse (autocomplétion, obligatoire)
    └── Téléphone (obligatoire, formaté XX XX XX XX XX)

complete-profile.tsx — Étape 2 / 2
    └── Nombre de places en voiture (liste déroulante 0–5)
        │ (validation)
```

```
        ▼
    Accueil de l'application
```

Parcours 4 — Trouver un trajet (3 taps)

```
Accueil
    │ tap "Je cherche"
    ▼
Liste des trajets disponibles
    │  Filtres : [Ponctuel] [Régulier]
    │ tap sur une carte de trajet
    ▼
Détail du trajet
    ├── Itinéraire, date, places restantes
    ├── Profil du conducteur (nom + rôle)
    └── [Demander une place]  ← tap 3
        │ Message optionnel au conducteur
        ▼
    Notification push + email au conducteur
```

Parcours 5 — Proposer un trajet (3 étapes)

```
Étape 1 / 3 — Départ et arrivée
    ├── Départ : [● Mon adresse] (pré-sélectionné)
    │           [○ Autre adresse] (autocomplétion)
    └── Arrivée : saisie libre (autocomplétion)

Étape 2 / 3 — Détails
    ├── Type : [Ponctuel] [Régulier]
    ├── Date (calendrier custom avec navigation mois/année)
    ├── Heure (formatée automatiquement HH:MM)
    ├── Nombre de places (1 à 5)
    └── Commentaire optionnel

Étape 3 / 3 — Récapitulatif
    └── [Publier le trajet] → visible immédiatement dans la liste
```

Parcours 6 — Validation des inscriptions (Admin)

```
(admin)/users.tsx
    │
    ├── Section "Demandes en attente"
    │     (comptes is_active = false, rôle non licencié)
    │     ├── [Valider]  → is_active = true, notification au parent
```

```
      │      └── [Refuser]  → suppression du compte
      │
      └── Section "Membres actifs"
            ├── Recherche par nom / numéro de licence
            └── Suspendre / réactiver
```

Parcours 7 — Création d'événement avec groupes ciblés (Admin)

```
(admin)/events.tsx
    │
    ├── Nom, lieu, date (début + fin)
    ├── Multi-jours, infos hébergement
    └── Groupes ciblés : chips sélectionnables
          ├── Aucun sélectionné → notifie TOUS les membres actifs
          └── Groupes sélectionnés → notifie uniquement ces groupes
```

Charte graphique

Élément	Valeur
Couleur principale	`#1A56DB` (bleu EGC)
Couleur accent	`#F59E0B` (or)
Succès	`#10B981` (vert)
Erreur	`#EF4444` (rouge)
Fond	`#F1F5F9` (gris clair)
Surface	`#FFFFFF`
Police	System (San Francisco / Roboto)
Border radius	12px (cards), 999px (chips)
Logo	`logo/logoNoirPng.png` (écrans de connexion)

---

9. Fonctionnalités avancées

Matching intelligent

L'algorithme de suggestion classe les trajets selon un score composite :

```
Score (sur 100) =
  40 pts × même groupe d'entraînement
+ 30 pts × proximité géographique (rayon 2 km)
```

```
+ 20 pts × trajet dans les 7 prochains jours
+ 10 pts × places disponibles
```

Implémentation : **fonction SQL PostgreSQL** `suggest_trips_for_user()` — aucun serveur applicatif nécessaire.

Notifications push et emails

Événement	Destinataire	Canal	Déclencheur
Nouvelle demande de place	Conducteur	Push + Email	Trigger INSERT bookings
Demande acceptée	Passager	Push	Trigger UPDATE bookings
Demande refusée	Passager	Push	Trigger UPDATE bookings
Annulation passager	Conducteur	Push + Email	Trigger UPDATE bookings
Nouveau message	Interlocuteur	Push	Trigger INSERT messages
Trajet annulé	Tous les passagers	Push	Trigger UPDATE trips
Nouvel événement	Membres des groupes ciblés	Push	Trigger INSERT events
Nouvelle demande d'inscription	Admins	Push	Appel RPC register_parent_account

Les emails sont envoyés via **Resend + pg_net** (extension PostgreSQL). Ils sont désactivables sans changement de code via `app_settings.email_notifications_enabled = 'false'`.

Feature flags — Fonctionnalités payantes

Toutes les fonctionnalités engendrant un coût sont contrôlées par des flags centralisés dans **deux endroits** :

`src/constants/features.ts` (côté app TypeScript) :

```
FEATURE_EMAIL_NOTIFICATIONS      = true  // ~0,80€/1 000 emails
FEATURE_AUTH_EMAIL_CONFIRMATION  = false // Quota 3/h sur plan gratuit
FEATURE_SMS_NOTIFICATIONS        = false // Non implémenté (~0,05€/SMS)
```

`app_settings` (côté SQL, modifiable sans redéploiement) :

```
UPDATE app_settings SET value = 'false' WHERE key = 'email_notifications_enabled';
```

Formatage automatique des saisies

Tous les champs de type date, heure et téléphone sont formatés automatiquement en temps réel :

Champ	Saisie	Résultat
Date de naissance	`22021996`	`22/02/1996`
Heure de départ	`1830`	`18:30`
Téléphone	`0612345678`	`06 12 34 56 78`

Implémentation : `src/lib/formatters.ts` — `formatDateInput`, `formatTimeInput`, `formatPhoneInput`.

Calendrier custom — DatePickerInput

Le sélecteur de date est un composant entièrement custom (sans dépendance native) :

- Grille de jours avec navigation mois par mois via les boutons `‹ ›`
- Tap sur le titre mois/année → sélecteur déroulant avec deux colonnes scrollables
- Support `minimumDate` / `maximumDate`
- Compatible iOS, Android et tous les SDK Expo sans configuration native

Adresse avec autocomplétion obligatoire

Le composant `AddressInput` utilise Nominatim (OpenStreetMap) :

- Suggestions en temps réel dès 3 caractères saisis
- Si l'utilisateur ne sélectionne pas un élément de la liste, le champ **s'efface automatiquement** à la perte du focus
- Empêche les adresses non vérifiées de passer en base

Logique de sécurité pour les enfants

- Tableau admin `enfant → conducteur` pour chaque événement
- Téléphone du conducteur affiché **uniquement** après confirmation de réservation
- Signalement en un tap → notification immédiate à l'admin
- Journal d'audit de toutes les actions sensibles

Trajets réguliers

- Règle de récurrence au format iCal (`RRULE:FREQ=WEEKLY;BYDAY=MO`)
- Occurrence générée à la demande (pas de cron job à maintenir)
- Passagers récurrents réinvités automatiquement chaque semaine

---

10. Roadmap produit

MVP — ✅ Livré (Juin 2026)

- Authentification dual-mode licencié / parent
- Inscription parent avec vérification de licence et validation admin
- Création et liste des trajets (Ponctuel, Régulier)
- Réservations avec validation conducteur + re-réservation après annulation

- Messagerie temps réel
- Notifications in-app (onglet dédié)
- Notifications push + emails transactionnels
- Interface admin (tableau de bord, membres, événements avec groupes ciblés)
- Feature flags pour les fonctionnalités payantes
- Formatage automatique des champs (date, heure, téléphone)
- Dépôt Git initialisé

V1.1 — Prochaines priorités

- Matching intelligent (suggestions personnalisées par score)
- Trajets réguliers avec récurrence
- Import CSV des licenciés (admin)
- Rappels automatiques avant chaque trajet (cron externe)
- Validation des associations parent-enfant depuis l'interface admin

V1.2

- Tableau de bord admin enrichi (statistiques détaillées)
- Système de signalement in-app
- Historique des trajets par membre
- Mode hors-ligne (consultation des trajets sauvegardés)

V2.0 — Long terme

- Carte interactive avec visualisation des itinéraires
- Export calendrier (iCal) des trajets confirmés
- Application web complémentaire (PWA)
- Statistiques d'usage pour le club
- Widgets iOS/Android (prochain trajet sur l'écran d'accueil)

---

11. Infrastructure et coûts

Architecture zéro maintenance

Tous les services sont **entièrement managés** — aucun serveur à administrer.

Service	Usage	Coût mensuel
Supabase Free	Base de données, Auth, Realtime, Storage	**0 €**
Expo EAS Free	Builds iOS/Android (30/mois), OTA updates	**0 €**
Expo Push Notifications	Notifications push illimitées	**0 €**
OpenStreetMap / Nominatim	Cartes et géocodage	**0 €**
Resend Free	3 000 emails/mois (emails transactionnels)	**0 €**
Sentry Free	Monitoring erreurs (5 000/mois)	**0 €**

Service	Usage	Coût mensuel
cron-job.org	Ping anti-pause Supabase (toutes les 72h)	**0 €**
GitHub	Dépôt privé (plan gratuit)	**0 €**
Apple Developer Program	Publication App Store iOS	**~8 €/mois**
Google Play Store	Publication Android	**25 € (×1)**

Total récurrent : ~8 €/mois Coût année 1 : ~120 € Coût années suivantes : ~99 €/an

Soit moins de 0,07 €/licencié/an.

Capacités du plan gratuit Supabase

Ressource	Limite	Usage estimé EGC
Stockage base de données	500 MB	~50 MB (1 600 users + trajets)
Stockage fichiers	1 GB	~160 MB (avatars)
Connexions Realtime simultanées	200	Pic estimé : 30-50
Utilisateurs actifs/mois	50 000	1 600
Emails Auth	3/heure	Non utilisé (confirmation désactivée)

Mesure anti-pause

Le plan Supabase gratuit met en pause les projets inactifs après 7 jours. Un ping automatique depuis **cron-job.org** (gratuit) toutes les 72 heures maintient le projet actif en permanence, y compris pendant les vacances scolaires.

Évolutivité

Si le club atteint 5 000 membres ou souhaite des backups quotidiens automatiques, la migration vers **Supabase Pro (25 €/mois)** se fait en un clic depuis le dashboard, sans aucun changement de code.

---

12. Recommandations

Bonnes pratiques appliquées

Sécurité

- Row Level Security sur toutes les tables (données isolées par utilisateur)
- Fonctions SECURITY DEFINER avec vérification d'identité explicite
- Jamais de donnée sensible exposée publiquement
- Tokens JWT à expiration courte + refresh tokens rotatifs
- Téléphone masqué jusqu'à confirmation de réservation
- Confirmation email Auth désactivée (évite le quota 3/h, validation manuelle suffisante)

Fiabilité

- Logique métier en base de données (triggers SQL) — pas de serveur custom à maintenir
- Services 100 % managés — pas de VPS, pas de patches de sécurité manuels
- Monitoring automatique (Sentry) — remontée des erreurs sans surveillance active
- Re-réservation via UPSERT — aucun blocage par contrainte unique

Simplicité utilisateur

- Maximum 3 taps pour une action principale
- Formulaire de création de trajet en 3 étapes guidées
- Adresse de départ "Mon adresse" pré-sélectionnée par défaut
- Formatage automatique des numéros de téléphone, dates, heures
- Messages d'erreur en français clair (rate limit, licence invalide, etc.)

Maintenabilité

- Feature flags centralisés pour activer/désactiver les coûts sans déploiement
- Code source versionné sur Git (dépôt privé GitHub)
- Migrations SQL numérotées et ordonnées (001 → 008)

Risques identifiés et mitigations

Risque	Probabilité	Impact	Mitigation
Inscription d'un non-membre	Faible	Critique	Vérification numéro de licence + validation admin
Compte admin unique compromis	Faible	Critique	Plusieurs comptes admin, MFA recommandé
Pause Supabase en vacances	Moyenne	Élevé	Ping automatique cron-job.org
Perte de données	Faible	Élevé	Export manuel mensuel (pg_dump via dashboard)
Abandon par les conducteurs	Moyenne	Élevé	Onboarding guidé, max 3 étapes pour créer un trajet
Harcèlement via messagerie	Faible	Élevé	Signalement en 1 tap, logs admin, blocage rapide
Limite emails Supabase Auth	Haute	Moyen	Confirmation email désactivée + validation manuelle

Ce qu'il ne faut pas faire

- Ne pas implémenter de géolocalisation en temps réel (disproportionné + RGPD)
- Ne pas ouvrir l'inscription au public sans validation admin
- Ne pas stocker les mots de passe en clair (Supabase Auth gère via bcrypt)
- Ne pas héberger soi-même sans équipe technique dédiée
- Ne pas exposer les numéros de téléphone dans les listes publiques
- Ne pas commiter le fichier `.env` sur Git (clés Supabase)

Checklist avant la mise en production

Migrations SQL (dans l'ordre)

- [ ] 001_initial_schema.sql
- [ ] 002_rls_policies.sql
- [ ] 003_functions_triggers.sql
- [ ] 004_email_notifications.sql
- [ ] 005_event_groups_notifications.sql
- [ ] 006_verify_license_function.sql
- [ ] 007_register_parent_function.sql
- [ ] 008_feature_flags.sql

Configuration Supabase

- [ ] Authentication → Email → décocher "Confirm email"
- [ ] `UPDATE app_settings SET value = 'ADRESSE_DU_CLUB' WHERE key = 'club_address'`
- [ ] `UPDATE app_settings SET value = 'CLE_RESEND' WHERE key = 'resend_api_key'`
- [ ] Variables d'environnement `.env` renseignées

Tests

- [ ] Compte administrateur créé et testé
- [ ] Ping anti-pause configuré sur cron-job.org
- [ ] Test de connexion avec un compte licencié (n° licence + date de naissance)
- [ ] Test d'inscription parent (vérification licence + demande admin)
- [ ] Test de validation admin d'un compte parent
- [ ] Test complet du parcours réservation (de bout en bout)
- [ ] Test annulation + re-réservation
- [ ] Test de la messagerie temps réel
- [ ] Test notifications push

Publication

- [ ] Compte Apple Developer activé (pour iOS)
- [ ] Compte Google Play créé (pour Android)
- [ ] Politique de confidentialité rédigée (obligatoire RGPD + stores)

Document mis à jour — Juin 2026 · Version 2.0 Architecture : React Native + Expo SDK 54 + Supabase Dépôt : GitHub (privé) — initialisé Juin 2026